

Часть 10

PSPACE-трудность и полнота. Примеры PSPACE-полных задач.

Опр Язык B называется PSPACE-трудным, если $\forall A \in \text{PSPACE} \cup A \leq_p B$

Зам Важно обратить внимание, что $\leq_p$ - сводимость по Карпу (т.е. полиномиальная сводимость по времени, а не по памяти).

Опр Язык B называется PSPACE-полным (обозначение $B \in \text{PSPACE}$), если $B \in \text{PSPACE}$ и B - PSPACE-трудный.

Примеры PSPACE-полных задач:

① $\text{SPACE-TSAT} = \{ \langle M, x, 1^s \rangle \mid M(x) = 1 \text{ и } M(x) \text{ занимает не более } s \text{ ячеек памяти} \}$

Теорема $\text{SPACE-TSAT} \in \text{PSPACE}$

□ 1) Сначала докажем, что $\text{SPACE-TSAT} \in \text{PSPACE}$. Нужно построить ЭМТ A , которая использует полиномиальную память и распознаёт этот язык. A будет моделировать работу M на слове x . Что надо хранить? Надо выделить s ячеек свободной памяти для работы на них M и ещё будем считать шаги. Мы уже доказывали, что если МТ использует $O(\text{poly}(x))$ ячеек памяти, то её время работы можно ограничить $2^{O(\text{poly}(s))}$ шагами, т.к. количество конфигураций такое. Для записи числа шагов в двоичной системе требуется $O(\text{poly}(\log(s)))$ памяти $\Rightarrow$ полное моделирование A тратит $O(\text{poly}(\log(s)))$ памяти $\Rightarrow$ построили необходимую ЭМТ $\Rightarrow \text{SPACE-TSAT} \in \text{PSPACE}$

2) Теперь докажем, что SPACE-TSAT - PSPACE-трудный.

Выберем произвольный $L \in \text{PSPACE}$. Для него $\exists M_L$ - ЭМТ, которая тратит не более

$p(x)$ ячеек памяти. Наша функция f по слову x будет выдавать тройку $(M_L; x; p(x))$. Из построения Π , что $x \in L \Leftrightarrow (M_L; x; p(x)) \in \text{SPACE} \text{MSAT}$.

Осталось доказать, что f вычислима за полиномиальное время. Выписывание M_L занимает $O(1)$ времени, т.к. не зависит от x . Выписывание x занимает $O(|x|)$ времени. Подсчет и запись $p(x)$ в унарной записи займёт $O(p(x))$ времени $\Rightarrow f$ вычисляется за полиномиальное время $\Rightarrow \forall L \in \text{PSPACE} \text{ и } L \leq_p \text{SPACE} \text{MSAT} \Rightarrow \text{SPACE} \text{MSAT} - \text{PSPACE-трудный}$.

т.к. $\text{SPACE} \text{MSAT} \in \text{PSPACE}$ и он P-SPACE-трудный $\Rightarrow \text{SPACE} \text{MSAT} \in \text{PSPACE} \text{ с } \blacksquare$

② $\text{SUCCINCTPATH} = \{(\varphi; u; v) \mid \text{в графе, построенном по формуле } \varphi \text{ есть путь из } u \text{ в } v\}$

Для начала поймаем формат входных данных. Для этого надо разобраться как граф задавать формулой. В графе G N вершин. Пронумеруем их, чтобы смог записать все вершины бинарным кодом, требуется $k = \lceil \log_2 N \rceil$ символов. Что же делает формула φ ? По заданным двум вершинам u и v в двоичной системе она выдаёт 1 тогда и только тогда, когда есть ребро между u и v . Т.е. $\varphi(u; v) = 1 \Leftrightarrow \text{в } G \text{ есть ребро между } u \text{ и } v$. Т.к. $u = u_1 u_2 \dots u_k$; $v = v_1 v_2 \dots v_k$, то φ - булева формула от $2k$ переменных. $|\varphi| = p(k)$, где p - полином. Таким образом входные данные полиномиальны от k .

Теорема $\text{SUCCINCTPATH} \in \text{PSPACE}$

□ 1) Сначала докажем, что $\text{SUCCINCTPATH} \in \text{PSPACE}$. То есть нам надо найти

проверить, есть ли путь между двумя вершинами в графе. Быстрее всего это делается с помощью dfs, то есть мы буквально его запустим на этом графе, осталось посчитать, за сколько он работает

Так как $|V| = N = \sum_{k=0}^{\infty} 2^k$, то $|E| = 2^{2k} \Rightarrow$ dfs работает за $O(|V| + |E|)$ времени $\Rightarrow O(2^k)$ времени. При этом нам надо использовать полиномиальную от k память.

Вспомним, что $PSPACE = NPSPACE$ и упростим себе жизнь, запровав это на НМТ.

То есть приходя в вершину, следующую мы будем выбирать недетерминированным образом. Заметим, что если есть путь из u в v , то его длина не более $|V|-1$, т.е. 2^k-1 . Тогда мы можем перебрать все пути длиной $\leq 2^k-1$. Будем каждый отрезок

пути обрабатывать в своей ветке. Что тогда хранится в ветке? Надо хранить счётчик (cnt), в который мы будем записывать число пройденных вершин $\rightarrow O(k)$

памяти. Пусть мы находимся в некой вершине x и выбираем следующую y .

Тогда если $y = v$, то выводим 1, если $cnt = 2^k-1$, а мы всё ещё не вышли, то пути в этой ветке нет $\Rightarrow$ возвращаем 0. После каждого выбора новой вершины

++cnt. Итого мы тратим $O(k)$ памяти на cnt, $O(k)$ на хранение нескольких нужных вершин, например, нам хватит x, y и v и $O(p(k))$ на вычисление

$\varphi(x, y)$ каждый раз $\Rightarrow$ каждая ветка НМТ работает за $O(p(k))$ по памяти $\Rightarrow$

$\Rightarrow$ $SUCCINCTPATH \in NPSPACE \Rightarrow SUCCINCTPATH \in PSPACE$

2) Теперь докажем, что $SUCCINCTPATH$ - $PSPACE$ -трудный. Выберем произвольный

$L \in PSPACE$. Тогда $\exists M_L$, которая решает $x \in L$ за $p(|x|)$ по памяти.

Тогда каждая конфигурация задаётся длиной $m \cdot \log(|\Sigma|)$, где m - константа, необходимая для кодирования в бинарную запись. Таблица δ NT M, можно считать заданной формулой для графа конфигураций, потому что мы можем перебирать символы и переходить по символу к новой конфигурации. Тогда δ по входу x выдаст M , начальную конфигурацию M на слове x и конечную принимающую конфигурацию (можно сделать из начальной и состояния q_{acc}). Тогда задача $x \in L$ эквивалентна тому, что есть путь от начальной конфигурации до принимающей в графе конфигураций. Осталось показать, почему δ полиномиально вычислима по времени. Потому что он передаёт константную NT, и переписывает две конфигурации, которые не надо вычислять. $\Rightarrow$ $\text{SUCCINCTPATH} \in \text{PSPACE}$ - трудный.

Т.к. $\text{SUCCINCTPATH} \in \text{PSPACE}$ и $\text{SUCCINCTPATH} \in \text{PSPACE}$ - трудный $\Rightarrow \text{SUCCINCTPATH} \in \text{PSPACE}$ ■

③ $GG = E(G, x)$ | в обобщённой игре города в графе G и начальной вершине x выигрывает 1 игрок }

Сначала надо понять формулировку. У нас есть конечное число городов - вершин графа

G . Если между вершинами u и v есть ребро, то после "города u " можно сказать

"город v ". Спрашивается, есть ли гарантированно выигрышная стратегия для первого

игрока. Выиграть - значит прийти в вершину, из которой нет ребра ни в одну из

несоседних вершин. Гарантированно выигрышная стратегия - значит, что при любом

ходе второго игрока, найдётся следующий ход, который далее приводит в победную

конфигурацию.

Теорема $GG \in PSPACE$

□ δ/g ■

④ $TQBF = \{ \varphi \mid \varphi - \text{выполнимая булева формула с кванторами} \}$

Теорема $TQBF \in PSPACE$

□ Доказательство нестрогое, но идея верная.

Заметим, что для языка GG мы можем написать условие принадлежности к языку, как

$\exists u_1 \forall u_2 \exists u_3 \dots \exists u_n: A(u_1; u_2 \dots u_n)$, где $A(u_1; u_2 \dots u_n)$ проверяет, что между u_i и

u_{i+1} есть ребро и n - число вершин. То есть мы записали предикатный язык в виде

булевой формулы с кванторами $\Rightarrow$ т.к. $GG \in PSPACE$, то $TQBF \in PSPACE$ ■